



Práctica 9. Clases abstractas e Interfaces

Objetivo

Diseñar clases que modelen entidades del mundo real aplicando correctamente la herencia e integrando el uso de clases abstractas e interfaces para definir jerarquías, comportamientos comunes y favorecer el polimorfismo, utilizando el lenguaje de programación orientada a objetos, con una actitud analítica, propositiva y con creatividad.

✂ Equipo y material de apoyo:

- Computadora Personal (PC)
- Ambiente de desarrollo o IDE (VSCODE)
- JDK Java Development Kit

DESARROLLO DE LA PRACTICA:

Programa 1: Clases Persona y Cliente

Diseñe una clase llamada Persona con campos para almacenar el nombre, la dirección y el número de telefono. Escriba uno o más constructores y los métodos de acceso y modificación adecuados para los campos de la clase.

A continuación, diseñe una clase llamada Cliente, que herede de la clase Persona. La clase Cliente debe tener un campo para el número de cliente y un campo booleano que indique si el cliente desea estar en una lista de correo. Escriba uno o más constructores y los métodos de acceso y modificación adecuados para los campos de la clase. Muestre un objeto de la clase Cliente en un programa sencillo.

Programa 2: Clases Cuenta Bancaria y Cuenta de Ahorros.

Diseñe una clase abstracta llamada Cuenta Bancaria para almacenar los siguientes datos de una cuenta bancaria:



- Saldo
- Número de depósitos este mes
- Número de retiros
- Tasa de interés anual
- Cargos mensuales por servicio

La clase debe tener los siguientes métodos:

Constructor: El constructor debe aceptar argumentos para el saldo y la tasa de interés anual.

Depósito: Un método que acepta como argumento el monto del depósito. El método debe sumar el argumento al saldo de la cuenta. También debe incrementar la variable que almacena el número de depósitos.

Retirar: Un método que acepta un argumento para el monto del retiro.

El método debe restar el argumento del saldo. También debe incrementar la variable que almacena el número de retiros.

Calcular Interés: Un método que actualiza el saldo calculando el interés mensual generado por la cuenta y sumándolo al saldo.

Esto se realiza mediante las siguientes fórmulas:

Tasa de Interés Mensual = (Tasa de Interés Anual / 12)

Interés Mensual = Saldo * Tasa de Interés Mensual

Saldo = Saldo + Interés Mensual

Proceso Mensual: Un método que resta los cargos mensuales por servicio del saldo, llama al método `Calcular Interés` y luego establece a cero las variables que almacenan el número de retiros, el número de depósitos y los cargos mensuales por servicio.



A continuación, diseñe una clase `Cuenta de Ahorros` que extienda la clase `CuentaBancaria`. La clase `Cuenta de Ahorros` debe tener un campo `status` para indicar si la cuenta está activa o inactiva. Si el saldo de una cuenta de ahorros cae por debajo de \$25, se vuelve inactiva. (El campo `status` podría ser una variable booleana). No se podrán realizar más retiros hasta que el saldo supere los \$25, momento en el que la cuenta se activará de nuevo.

La clase `Cuenta de Ahorros` debe tener los siguientes métodos:

`Retirar`: Un método que determina si la cuenta está inactiva antes de realizar un retiro. (No se permitirá ningún retiro si la cuenta no está activa). El retiro se realiza llamando a la versión de la superclase del método.

`Depositar`: Un método que determina si la cuenta está inactiva antes de realizar un depósito. Si la cuenta está inactiva y el depósito eleva el saldo por encima de \$25, la cuenta se activará de nuevo. El depósito se realiza llamando a la versión de la superclase del método.

Antes de llamar al método de la superclase, este método verifica el número de retiros realizados. Si el número de retiros del mes supera los 4, se añade un cargo por servicio de \$1 por cada retiro adicional al campo de la superclase que almacena los cargos mensuales.

(No olvide verificar el saldo de la cuenta después de aplicar el cargo por servicio. Si el saldo cae por debajo de \$25, la cuenta se desactiva).

Programa 3: Clases Ship, CruiseShip y CargoShip

Diseñe una clase Ship con los siguientes miembros:

- Un campo para el nombre del barco (cadena de texto).
- Un campo para el año de construcción del barco (cadena de texto).
- Un constructor y sus correspondientes métodos de acceso y modificación.
- Un método toString que muestre el nombre del barco y el año de construcción.



Diseñe una clase `CruiseShip` que extienda la clase `Ship`. La clase `CruiseShip` debe tener los siguientes miembros:

- Un campo para el número máximo de pasajeros (entero).
- Un constructor y sus correspondientes métodos de acceso y modificación.
- Un método `toString` que sobrescribe el método `toString` de la clase base. El método `toString` de la clase `CruiseShip` debe mostrar únicamente el nombre del barco y el número máximo de pasajeros.

Diseñe una clase `CargoShip` que extienda la clase `Ship`. La clase `CargoShip` debe tener los siguientes miembros:

- Un campo para la capacidad de carga en toneladas (un entero).
- Un constructor y sus correspondientes métodos de acceso y modificación.
- Un método `toString` que sobrescribe el método `toString` de la clase base. El método `toString` de la clase `CargoShip` debe mostrar únicamente el nombre del barco y su capacidad de carga.

Demostremos el funcionamiento de las clases en un programa con un array de `Ship`. Asigne varios objetos de tipo `Ship`, `CruiseShip` y `CargoShip` a los elementos del array. El programa debe recorrer el array, llamando al método `toString` de cada objeto.

Entrega:

- La entrega se realizará mediante una actividad en Classroom. Se han de entregar los archivos fuentes desarrollados, más el reporte de la practica adjuntos adjuntos en un fichero zip cuyo nombre esté formado por el número de práctica y nombre del alumno. Ejemplo `Practica8_JuanLopez.pdf`
- Hacer una clase por cada ejercicio y adjuntarlos



- Entregar un reporte de práctica, formato reporte técnico y electrónico, podrá utilizar este formato y agregar el procedimiento correspondiente), que conceptos fundamentales se aplicaron en esta practica, junto con resultados y conclusiones.

Análisis de Resultados

Conclusiones

Referencias

Criterios para evaluar:

Código (70 puntos)

- Debe incluir comentarios en su programa, sin llegar al uso exagerado de ellos.
- Debe utilizar sangrías en los lugares apropiados.
- Poner como encabezado del programa en comentarios, titulo, la descripción de la práctica, nombre del alumno que realizo la práctica.
- Funcionamiento correcto del programa

Reporte (30 puntos):

- | | |
|---|------------|
| • Portada, competencia u objetivo | 2.5 puntos |
| • Pegar Código del programa (anexar archivos .java) | 2.5 puntos |
| • Explicación completa del código y su funcionamiento | 15 puntos |
| • Resultados (análisis de resultados) | 5 puntos |
| • Conclusiones: | 5 puntos |

Nota: No se aceptarán trabajos fuera de la fecha de entrega o que no cumplan con las especificaciones de entrega requeridas. Los trabajos que se detecten copiados, serán anulados automáticamente para ambos alumnos.